

Frontend Client — Student Portal

React 18 SPA architecture, Vite 8 build pipeline, Three.js 3D canvas, Framer Motion animations, and responsive mobile-first design system.

Framework: React 18 + Vite 8 3D: Three.js WebGL Canvas Mobile-First Responsive

1. Frontend Architecture Overview

REACT COMPONENT ARCHITECTURE & RENDERING LAYERS

BROWSER

App.jsx (React Router v6 — 8 Routes)

/Home

HeroSection+3D

/universities

Search + Filters

/ai-counselor

Groq Chat UI

/scholarships

Eligibility Calc

/connect

Lead Form CTA

/student

Dashboard

Shared UI Components Layer

Navbar • PillButton • TestimonialCard • ScholarshipCard • AIChat • FinalCTA

State: React Hooks (useState / useEffect)

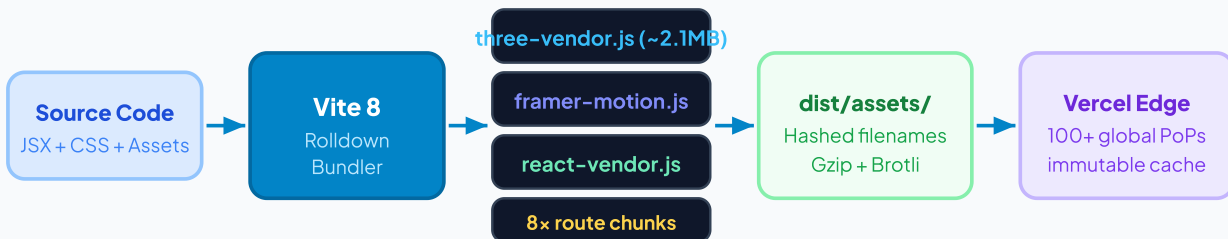
No Redux — local state + prop drilling for simple flows

Vite Build: Manual Chunk Splitting

three-vendor • framer-motion • react-vendor • route-chunks

Data: Axios → Backend REST API (CORS-whitelisted) → MongoDB Atlas

VITE 8 PRODUCTION BUILD PIPELINE — CODE SPLITTING & CHUNK STRATEGY



2. Technology Decision Framework

1. React 18 + Vite 8 vs Next.js SSR

FRAMEWORK

WHAT IS IT?

A client-side rendered Single Page Application (SPA) using React 18 with concurrent features, built and bundled by Vite 8 using the Rollup Rust-based bundler.

HOW DOES IT WORK & PROBLEM SOLVED?

React Router v6 provides client-side navigation with zero full-page reloads. Dynamic imports via `React.lazy()` ensure heavy pages (Three.js globe, AI chat) only download when visited, reducing initial load to <195kB gzipped.

WHAT ALTERNATIVES EXISTED?

Next.js 14 (SSR): Better SEO by default, but requires Node.js server for SSR — adding backend complexity. **Create React App:** Webpack-based, 10x slower builds.

WHY DID WE CHOOSE THIS APPROACH?

Marketing pages use static pre-rendered HTML (SEO via meta tags). The real UX value is in authenticated interactive dashboards — where SSR provides negligible benefit but adds significant operational cost.

2. Three.js WebGL vs CSS 3D Animations

3D RENDERING

WHAT IS IT?

Hardware-accelerated 3D WebGL scenes embedded in the hero section and FinalCTA banner using Three.js.

HOW DOES IT WORK & PROBLEM SOLVED?

WebGL renders on the GPU compositor, bypassing the browser's CSS layout and paint phases entirely. The globe, particle systems, and 3D objects run at 60fps without causing layout jank on the main thread.

WHAT ALTERNATIVES EXISTED?

CSS 3D transforms: Limited to flat planes, no real depth, no particle systems. **Lottie animations:** 2D SVG-based, no WebGL depth effects.

WHY DID WE CHOOSE THIS APPROACH?

Differentiates UniCoach visually in a commoditized market of cookie-cutter education platforms. The immersive 3D experience increases session duration and perceived product quality.

💡 Senior Interview Defense: "Describe your frontend architecture decisions"

"For UniCoach's student portal, I architected a React 18 SPA using Vite 8 with strategic code splitting — Three.js, Framer Motion, and React itself ship as separate cached vendor chunks. Route-level lazy loading reduces initial download to under 195kB gzipped. Three.js WebGL scenes on the hero run at 60fps via GPU compositing. For mobile, I applied responsive breakpoints with hidden classes to prevent avatar and element collisions on sub-640px viewports."